select algorithm

mahmood

February 24, 2023

an algorithm that would help us reduce the time complexity of many problems
input: an array A of n numbers and a normal number k such that $1 \leq k \leq n$
output: the element in A whose rank is k, i.e. the k'th smallest number in A
example: for the input $k = 3, A = \{13, 15, 16, 14, 12, 18, 17, 11\}$, the output is 13
a naive approach:

```
NAIVE(A, k):
    A ← SORT(A)
    return A[k]
```

the correctness of this algorithm trivially follows from the definition of the given problem
we propose a more afficient algorithm that we'll call SELECT which solves the problem in linear time:

```
SELECT(A, k):
    if |A| < 10 then
        return FIND(A, k)
    p ← WISE-CHOOSE-PIVOT(A)
    [A_l, A_r] ← PARTITION(A, p)
    m ← |A_i|
    if k ≤ m then
        return SELECT(A, k)
    else
        return SELECT(A, k-m)
FIND(A, k):
    A ← BUBBLE-SORT(A)
    return A[k]
WISE-CHOOSE-PIVOT(A):
    for i ← 1 to ⌈n/5⌉ do
        for j ← 1 to 5 do
            B[j] ← A[(i − 1) · 5 + j]
        B ← SORT(B)
        medians[i] ← B[3]
    return SELECT(medians, ⌊|medians|/2⌋ + 1)
```

Lemma 1. given an array A of n different numbers, and a number k, $1 \leq k \leq n$, let e_k be a number such that $\text{rank}(e_k, A) = k$
assume that the partition algorithm was applied to A and we got the subarrays A_l, A_r of sizes $m, n - m$

1. if $k \leq m$ then e_k is in A_l and $\text{rank}(e_k, A_l) = k$

2. if $k > m$ then e_k is in A_r and $\text{rank}(e_k, A_r) = k - m$

for example, given the array $A = \{3, 8, -2, 13, 11, 6, 9, 2, 14, 7\}, pivot = 8, k = 6$
we'd get $A_l = \{3, -2, 6, 2, 7\}, A_r = \{8, 13, 11, 9, 14\}$
in A_l there are the 5 smallest numbers from A, the sixth smallest number in A, 8, is the minimal number in A_r

the correctness of the algorithm follows trivially from the lemma

note

we are sacrificing linear time finding a good pivot so that the partition algorithm's worst-case performance doesnt reach $\Theta(n^2)$, which may happen when the size of one of the partitions happens to be 1 repeatedly

Example 0.0.1.

Input: an array A
Output: numbers that appear in the array A more than $\lceil \frac{n}{4} \rceil$ times

$m_1 \leftarrow \textsc{Select}(A, \lceil \frac{n}{4} \rceil)$
$m_2 \leftarrow \textsc{Select}(A, \lceil 2\frac{n}{4} \rceil)$
$m_3 \leftarrow \textsc{Select}(A, \lceil 3\frac{n}{4} \rceil)$
$u_1 \leftarrow \textsc{Count}(A, m_1)$
$u_2 \leftarrow \textsc{Count}(A, m_2)$
$u_3 \leftarrow \textsc{Count}(A, m_3)$
return of u_1, u_2, u_3 the ones that are bigger than $\lceil \frac{n}{4} \rceil$

this algorithm runs in linear time

Example 0.0.2. a company has n employees, their salaries differ and are given in an array A, it wants to give its employees bonuses (it wants to spend as much as it can, given a limited budget), a bonus is the same amount as ones salary, given k, the sum of bonuses is less or equal to a given number k, solve the problem in $\Theta(n)$

```
FIND(A, k):
    s ← ∑A
    if s ≤ k then return A
    a ← SELECT(A, ⌊n/2⌋)
    (A_s, sum_s) ← SUM-SMALL(A, a)  // elements smaller than a
    (A_b, sum_b) ← SUM-BIG(A, a)  // elements bigger than a
    if sum_s > k then return FIND(A_s, k)
    if sum_s = k then return A_s
    return A_s ∪ FIND(A_b, k − sum_s)
```

the recurrence formula for this algorithm is $T(n) = Cn + T\left(\frac{n}{2}\right)$ which simplifies to $T(n) \in \Theta(n)$